reduce/kary

An AgentMPI collective scaling analysis

Abstract

The k-ary reduction is the algorithm the agent setting actually wants and the one MPI would never choose: a tree of arity k in which each interior rank combines all its children in a *single* operator application, using the operator's variadic kernel. It sends the same $p-1$ messages as every other reduction, but its fold depth is $\lceil \log_k p \rceil$ rather than $\lceil \log_2 p \rceil$ and — the part the closed form omits — its application count is $\lceil (p-1)/(k-1) \rceil$ rather than $p-1$. **This family has no sweep.** The 500-run archive contains 450 collective sweep runs and the `reduce` sweeps cover only `flat`, `binomial` and `chain`, so `analyze_family.py` builds no package for it and this document has no `generated.tex`, no `metrics.json` and no family figure. What it does have is four archived runs that exercise the algorithm at $p = 8$ and $p = 16$ with $k = 4$, and they confirm the closed form on every axis it predicts: 7 and 15 messages, 2 rounds, fold depth 2, `variadic: true` at every interior rank, and 3 and 5 operator applications where $\lceil (p-1)/(k-1) \rceil$ says 3 and 5. They also price it: at $p = 8$ with a real agent operator, k-ary took 51.8 and 53.1 s of root blocking against the binary tree's 80.3 and 77.6, the flat reduction's 251.6 and 197.6, and the chain's 634.4 and 1,377.0, at half the binary tree's output tokens. The single most important finding, though, is a defect: `reduce(algorithm="kary")` does *not* carry the guard that `binomial` carries, so with a non-commutative operator at a non-zero root it silently evaluates in rotated rank order. A four-rank probe at `root = 1` returns `'[1][2][3][0]'` where rank order demands `'[0][1][2][3]'`, while `binomial` refuses the identical call. That is exactly the silent quality regression the design says the runtime exists to prevent.

1 What this family is, and why it has no sweep

Every other family in this set is one algorithm measured at 21 process counts. This one is not measured at any, and the reason is mechanical rather than interesting. `scripts/analyze_family.py` discovers families by matching the regular expression `coll-([a-z]+)-(.+)-(\d+)$` against the names in `traces/manifest.json`. The manifest holds 500 runs, of which 450 match that pattern, and the `reduce` sweeps among them use exactly three algorithms:

`{re.match(...).group(1) for name in manifest if "coll-reduce-" in name}`	
reduce sweep algorithms present	`binomial`, `chain`, `flat`
runs matching `coll-reduce-kary-*`	none

So `python3 scripts/analyze_family.py -family reduce/kary` prints `unknown family: reduce/kary` and writes nothing. There is no `metrics.json`, no `generated.tex` and no `figures/family.pdf` for this family, and rather than fabricate them I am recording the absence. Every number in what follows is sourced: to an archived event log under `traces/events/`, to the implementation in `src/`, or to a probe whose exact command is stated.

The absence is a gap in the sweep design rather than in the implementation. `kary` is a first-class algorithm in `algorithms.reduce`, it has an entry in `cost.FORMULAS`, it has a dedicated predictor (`cost.predict_kary`), it appears in the paper's tabulated cost formulas, and it has four dedicated tests. It is the algorithm the paper argues for. Not sweeping it means the one quantitative claim the sweep exists to make — measured counts against closed forms across every process count — was never made for the algorithm the paper recommends, and in particular the non-power-of-two question that motivates the sweep's marker classes has been asked of `binomial` at thirteen sizes and of `kary` at two.

What the archive does contain

Four archived runs execute `reduce` with `algorithm="kary"`, all from the reduction-fidelity microbenchmark (`experiments/microbench.py`, `bench_fidelity`), all with $k = 4$ and `root = 0`. Reading their `coll.reduce` and `agent.call` events directly:

run	operator	p	k	msgs	rounds	depth	applic.	root wall
`sat-mb-fidelity__fidelity-kary-f-k4`	agent	8	4	7	2	2	3	53
`inc-mb-fidelity-inc__fidelity-kary-f-k4-inc`	agent	8	4	7	2	2	3	51
`fid-inc-smoke__fidelity-kary-f-k4-inc`	function	8	4	7	2	2	3	0.0
`fid-synth__fidelity-kary-f-k4`	function	16	4	15	2	2	5	0.0
closed form, $p = 8$, $k = 4$				$p-1 = 7$	$\lceil \log_4 8 \rceil = 2$	2	$\lceil 7/3 \rceil = 3$	
closed form, $p = 16$, $k = 4$				$p-1 = 15$	$\lceil \log_4 16 \rceil = 2$	2	$\lceil 15/3 \rceil = 5$	

"msgs" is the count of `msg.send` events in the log; "rounds" and "depth" are the maxima of the `rounds` and `fold_depth` fields over the p `coll.reduce` records; "applic." is the count of `agent.call` events, which for this benchmark is exactly the number of operator applications because the operator's only action is one invocation; "root wall" is the `wall_s` field of rank 0's `coll.reduce` record. Every measured value matches its closed form, and `variadic: true` appears in the `coll.reduce` extras of every interior rank in all four runs, confirming the wide path was taken rather than the degraded one.

2 How the algorithm scales

The implementation is a level-synchronous k-ary tree over virtual ranks. Each rank computes $vr = (r - root) \bmod p$ and walks `stride` through $1, k, k^2, \ldots$ while `stride < p`. At each level it computes $group = (vr/stride) \bmod k$; if that is non-zero it sends its accumulator to $vr - group \cdot stride$ and stops, and if it is zero it receives from every $vr + j \cdot stride$ for $j = 1 \ldots k-1$ that is less than p, and then calls

```
acc = op.combine([acc, *incoming], ctx)
```

once. That single line is the whole idea. `Op.combine` dispatches to the variadic kernel when one exists, so k children cost one application at one level of depth; without a variadic kernel it degrades to a left fold that costs $k - 1$ successive applications, which is exactly the depth the wide tree was meant to save, and the implementation records `extra["variadic"]` so that the degradation is visible rather than silent.

Three quantities follow, and they scale differently from one another:

Messages: $p-1$, independent of k. Every non-root virtual rank sends exactly once, at whichever level first gives it a non-zero group index. So widening the tree is free in traffic — which is the claim that makes the whole argument work, and is why `test_wider_fanin_reduces_fold_depth_at_equal_message_co` asserts `observed[2][0] == observed[8][0]`. Measured: 7 at $p = 8$ and 15 at $p = 16$.

Rounds and fold depth: $\lceil \log_k p \rceil$. The stride loop runs $\lceil \log_k p \rceil$ times, computed integrally by `_ilogk_ceil` so it is exact at the boundaries, and because each level is one combine the fold depth equals the round count. This is where k earns its keep, and the effect is large: computing `cost.predict_kary(p, n, k).fold_depth` over a grid gives, at $p = 64$, depths of $6, 4, 3, 2, 2, 2$ for $k = 2, 3, 4, 8, 16, 32$. Going from $k = 2$ to $k = 8$ at $p = 64$ takes the depth from six to two at identical message count.

Applications: $\lceil (p-1)/(k-1) \rceil$. This is the quantity the closed form does not carry, and it is the one the price depends on. Each interior node at each level performs one application and absorbs $k-1$ senders, so the number of applications is the number of (node, level) pairs with at least one child. At $p = 8$, $k = 4$ that is three: ranks 0 and 4 combine three children each at `stride = 1`, and rank 0 combines its one remaining child (rank 4) at `stride = 4`. At $p = 16$, $k = 4$ it is five: four parents at `stride = 1` and rank 0 again at `stride = 4`. Both figures are confirmed by the `agent.call` counts in the table above — 3 and 5 — against $p - 1 = 7$ and 15 for every other reduce algorithm. Under a lossy operator this is a 2.3-fold and 3-fold reduction in the number of times a model is asked to re-summarise anything.

The two viewer screenshots make the structure legible in a way the counts do not. `sat-mb-fidelity__fidelity-kar` ($p = 8$, real agents) shows a 53.1 s span with three blue "agent working" bars and nothing else of substance: rank 4 works from roughly 3 to 25 s, and rank 0 works from about 5 to 28 s and again from 30 to 53 s. So the two `stride = 1` combines run *concurrently* — rank 0's first fold overlaps rank 4's — and only the top-level combine is serialised behind them. The stats row reports 3 agent calls, 7 messages, 4.4k tokens in and 1.6k out, \$0.037, and an agent latency p50/p95 of 25.5/25.5 s; the collectives panel reports max rounds 2, messages 7, max fold depth 2. Ranks $1, 2, 3, 5, 6, 7$ carry nothing but a lifecycle tick and a message tick, which is the correct picture of six leaves that contribute and stop. `fid-synth__fidelity-kary-f-k4` ($p = 16$, surrogate operator) shows the same shape at the next size up: 5 agent calls, 15 messages, max rounds 2, max fold depth 2, with activity concentrated in ranks $0, 4, 8, 12$ — precisely the four `stride = 1` parents — and rank 0 closing last.

3 Powers of two and everything else

For this algorithm the interesting boundary is not powers of two but powers of k, and the two measured points happen to straddle it. At $p = 16$ with $k = 4$ the tree is complete: every parent at `stride = 1` has exactly three children and rank 0's three top-level children $(4, 8, 12)$ all exist. At $p = 8$ with $k = 4$ it is not: rank 0's top-level children would be $4, 8, 12$ and only 4 is less than p, so the top level is a single-child combine. The remainder handling is the guard `if vr + j*stride < p` inside the children comprehension, and it behaves — $p = 8$ gave 7 messages, 2 rounds, fold depth 2 and 3 applications, all as predicted for an incomplete tree. That is one measured non-power-of-k size, against thirteen for each of the swept families, and it is the weakest part of this document's evidence.

What partially compensates is the test suite, which covers sizes the archive does not. `test_wider_fanin_reduces_fo`

runs $p \in \{8, 16, 27, 32\}$ against $k \in \{2, 4, 8\}$ and asserts both `msgs == size - 1` and `depth == _logkc(size, fanin)` at every combination, which includes $p = 27$ — neither a power of two nor a power of 4 or 8 — and `test_kary_reduce_matches_serial_fold` checks the result against the reference left fold for $k \in \{2, 3, 4, 8\}$. Those are assertions in a suite rather than measurements in the archive, and the distinction matters for a document whose job is to report what was measured, but they are the reason I would not expect the remainder path to be broken.

4 A real defect: `kary` lacks the non-commutative guard that `binomial` has

This is the most consequential thing in this document and it is a bug in AgentMPI itself.

`algorithms.reduce` contains an explicit guard, with a comment explaining exactly why it is needed. A tree keeps the lower *virtual* rank as the accumulator at every combine, so rank order is preserved along each path — but virtual rank order coincides with communicator rank order only when the root is 0. For any other root a non-commutative operator would be evaluated in rotated order, so the runtime refuses:

```
if algorithm == "binomial" and not op.commutative and root != 0:  raise AmpiUsageError(...)
```

The condition names `binomial`. `kary` performs the identical rotation — $vr = (r - root) \bmod p$, with `combine([acc, *incoming])` ordered by ascending virtual rank — and is not covered. I verified the consequence with a probe run entirely under `/tmp`, touching no repository artefact, using `CONCAT` (associative, EXACT, and declared *not* commutative) at $p = 4$ with `root = 1`:

algorithm	result at `root = 1`	expected	verdict
`chain`	`'[0][1][2][3]'`	`'[0][1][2][3]'`	order preserved
`flat`	`'[0][1][2][3]'`	`'[0][1][2][3]'`	order preserved
`binomial`	`AmpiUsageError`: cannot preserve rank order…		refused
`kary`, $k = 4$	`'[1][2][3][0]'`	`'[0][1][2][3]'`	**order violated**
`kary`, $k = 2$	`'[1][2][3][0]'`	`'[0][1][2][3]'`	**order violated**

The rotation is visible in the output: with `root = 1` the virtual ranks are $vr(1) = 0, vr(2) = 1, vr(3) = 2, vr(0) = 3$, and the result is the contributions in exactly that order. The call succeeds, returns a well-formed answer, emits no warning, and sets no `divergence_risk` flag. `kary`, $k = 2$ is a binary tree in every respect that matters and is silently wrong where `binomial` — the same shape by another name — is loudly refused.

Two things make this worse than a missing validation. First, it defeats the design principle the module states in its own docstring: an operator's declared algebra constrains algorithm selection, and "requesting a tree for such an operator is an error rather than a silent quality regression". Here it is precisely a silent regression. Second, the operator most likely to hit it is the one this repository actually uses. The reduction-fidelity benchmark constructs `MERGE_REPORTS_FIXED` with `commutative=False` and `associativity=APPROX`; it escapes the bug only because `bench_fidelity` calls `comm.reduce(..., root=0, ...)`. A harness that placed its integrator anywhere but rank zero — which is the natural thing to do when rank zero is the planner — would get an error from `binomial` and a rotated answer from `kary`, and would have no way to tell that the second had happened.

The test that would have caught it exists and is one parameter short. `test_kary_reduce_preserves_rank_order` checks exactly this property, over sizes $4, 8, 16$ and fan-ins $2, 4, 8$ — all at `root=0`. Adding `root` to its parametrisation, or changing the guard's condition to `algorithm in ("binomial", "kary")`, closes it. I have not made either change: this is an analysis document and the instruction is not to modify `src/` or `tests/`.

A second, smaller defect: the price model charges $p - 1$ applications

`cost.predict` computes a reduction's price as `message_price(volume, (p-1) * op_cost_tokens)`, and `cost.predict_kary` does the same with the same $(p - 1)$ factor. For every other reduce algorithm that is correct — `flat`, `chain` and `binomial` all perform $p - 1$ applications. For `kary` it is wrong by a factor of about $k - 1$: the measured application counts are 3 at $p = 8$ and 5 at $p = 16$ with $k = 4$, against 7 and 15. The paper's own `tab:reduce-alg` states the right figure — "$p - 1$ transfers, $\lceil \frac{p-1}{k-1} \rceil$ applications" — so the code contradicts the table it is meant to support. The direction of the error is conservative, in that it understates k-ary's advantage rather than overstating it, but it understates the paper's central recommendation on the axis the paper cares most about, and it means `best_algorithm(op, p, n, objective="price")` cannot see the saving at all (it returns `binomial` at every p I tested). The measured output-token counts show the saving is real: 1,576 and 1,323 tokens under `kary` against 2,903 and 2,492 under `binomial` at $p = 8$.

For completeness: the clamp that breaks `flat`'s predicted *time* — `min(op_rounds, rounds)`, discussed in the `reduce/flat` analysis — does not bite here, because `kary`'s rounds and fold depth are the same integer, and `predict_kary` does not clamp at all.

5 When to choose this algorithm

Whenever the operator has a variadic kernel, and the fan-in should be the widest the receiving rank's context admits. That is the opposite of MPI's preference and the reasoning is worth stating precisely, because it is the sharpest technical claim in this repository. Every $\log_2 p$ in MPI's collective layer descends from processes being *single-ported*: two incoming messages cost two transfers whatever the topology, so a wider node buys nothing and loses to serialisation. An agent rank has no port count. A prompt carries eight artifacts as easily as two, and a variadic operator merges eight in one call rather than seven. The binding constraint becomes context capacity, which is what `ops.optimal_fanin` computes: $k^* = \lfloor C(1 - h)/s \rfloor$ for budget C, per-input size s and reserve fraction $h = 0.25$, capped at 32. Evaluating it, a 128k-token budget with 1k-token payloads gives $k^* = 32$; with 4k-token payloads, 24; a 32k budget with 1k payloads gives 24; an 8k budget with 1k payloads gives 6. The cap exists because the relationship is not monotone in practice — a model handed fifty documents in one call attends to them unevenly — and where the cap belongs is stated in the source as an open empirical question.

Against the alternatives at the one size where all four were measured with a real agent operator ($p = 8$, from `sat-mb-fidelity` and `inc-mb-fidelity-inc`):

algorithm	rounds	depth	applications	root wall (s)	output tokens
`chain`	7	7	7 / 7	1,377.0 / 634.4	3,855 / 2,623
`flat`	1	7	7 / 8	197.6 / 251.6	3,456 / 3,372
`binomial`	3	3	7 / 7	77.6 / 80.3	2,903 / 2,492
`kary`, k=4	2	2	**3 / 3**	**53.1 / 51.8**	**1,576 / 1,323**

The one figure in that table that is not $p-1$ or $\lceil (p-1)/(k-1) \rceil$ is `flat`'s eight calls in the second campaign, and it is instructive rather than anomalous: the log holds seven `agent.call` events and one `agent.contract_violation` recording "`payload:  472 tokens exceeds max_tokens=450`" at `merge:d3`, followed by a successful attempt 2. The retry cost 92.0 s against a per-call median of 25 s, so a $p-1$-deep serial fold at the root is also the arrangement that gives a single overrun the most room to hurt.

k-ary wins on every axis in both campaigns: 1.5 times faster than the binary tree, 4–5 times faster than `flat`, 12–26 times faster than `chain`, and roughly half the binary tree's output tokens because it performs three applications instead of seven. It sends the same seven messages as all three. The `merge:d`n labels on the `agent.call` events make the depth visible directly: `flat` and `chain` emit `merge:d1` through `merge:d7`, `binomial` reaches `d3`, and `kary` reaches `d2` — and because the operator's output budget is $\max(300, B(1+0.25d))$, the deeper folds are also the more expensive ones. In the non-capacity-bound campaign `flat`'s seven applications emitted $354, 425, 373, 454, 535, 617$ and 698 tokens as the depth climbed, which is where its 3,456 total comes from. There is no crossover to report — no size or payload in the archive at which another reduce algorithm beats it — and that is unsurprising, because widening the tree costs nothing in traffic and the only thing it can cost is context occupancy, which no archived run exercises (the reduce path receives with `admit=False`, and the four kary runs report `occupancy:  0.0` and `admissions:  0` at every rank).

The one honest qualification concerns *why* it wins, and here the repository's own measurement corrects the paper's original argument. `kary` was proposed on fidelity grounds: shallower folds should preserve more of a lossy operator's input, with $F(d) = (1-\delta)^d$ giving 0.902 at depth 2 against 0.735 at depth 6 for $p = 64$, $\delta = 0.05$. The fidelity experiment refutes that, and the two campaigns in the table above are the two halves of the refutation. Under `inc-mb-fidelity-inc`, where the payload was made incompressible and the 450-token budget was enforced by contract, retention is 0.3854 for *all four* algorithms — at fold depth 7, at 3 and at 2, with 37 of 96 items surviving, zero mangled and a rank-position correlation of -0.8629 in every case. Under `sat-mb-fidelity`, where the budget was not binding, retention is 1.0000 for all four. Depth changed nothing in either regime. Under a uniform enforced budget the last application is the binding one, so a deep tree discards at intermediate levels and a shallow one discards at the root and both arrive at the same place; losses do not compound with depth because there is no slack to compound into. So k-ary's case is cost, decisively and measurably, and its fidelity case is a conditional that depends on intermediate nodes being allowed larger budgets than the root — which is what the operator interface's `weight` field exists to enable and what no archived run uses. That the recommendation survives on firmer ground than it was proposed on is the more useful result.

6 Threats to this reading

The largest threat is the one stated at the top: **there is no sweep**. Every other family in this set rests on 21 measurements across thirteen process counts, two independent campaigns, and an automatic check of the collective's self-report against the fabric's transport log at every size. This one rests on four runs at two process counts with one fan-in. The closed form is confirmed where it was measured and is unmeasured everywhere else: I cannot say whether `kary` logs $p-1$ messages at $p = 20$, whether its round count steps where $\lceil \log_k p \rceil$ says it should, or whether its self-reported count agrees with the log at any size other than 8 and 16. Nor is the family's own accounting checked as the swept families' are: I verified $7 = 7$ and $15 = 15$ by hand from the logs, which is not the same as 21 automatic comparisons.

Only one fan-in was ever measured, and it is not the one the model recommends. All four runs use $k = 4$. The tabulated cost entry uses `DEFAULT_FANIN = 8`, and `optimal_fanin(128000, 1000)` returns 32. So the fan-in the paper's policy would choose is eight times the fan-in that was measured, and the failure mode that matters at large k — a model attending unevenly to thirty-two inputs in one prompt, which is the reason the cap exists — is exactly the regime with no data. Every claim I make about wide trees is extrapolated from $k = 4$ by way of a closed form.

Two of the four runs use a surrogate operator rather than a model. `fid-inc-smoke__fidelity-kary-f-k4-inc` and `fid-synth__fidelity-kary-f-k4` have sub-second walls (0.026 and 0.053 s) because the operator is a Python function; they tell me about the protocol's counts and nothing about agent behaviour. Only `sat-mb-fidelity` and `inc-mb-fidelity-inc` ran against real workers via the broker executor, and those are two runs at one size, so the latency comparison in the previous section is a comparison of two samples per algorithm, not a distribution. Their per-application latencies (25.5 and 31.0 s p50) are of the same order across all four algorithms, which is why the wall-clock ordering tracks the application count so cleanly, but a campaign in which one algorithm happened to draw slower workers would muddle that, and I cannot rule it out from two runs.

The application-count claim rests on an identification I should make explicit. I counted `agent.call` events and called them operator applications. That is valid for this benchmark because `MERGE_REPORTS_FIXED`'s kernel does exactly one invocation per call and the runs contain no other agent work — 3 calls, 3 `broker.enqueue`, 3 `broker.claim` and 3 `broker.complete` in the $p = 8$ agent runs. It would not be valid for an operator that retried, or for a run with agent work outside the reduction.

Finally, one reading of the viewer needs correcting rather than reporting. The rank-health table in `sat-mb-fidelity__fidelity-kary-f-k4`'s screenshot shows ranks 1 and 2 in red, state `running`, alive `no`, suspected `fail_stop`. Nothing failed: the run's `metrics.json` reports `ok: true`, `failed_ranks: []`, `n_trouble: 0`, `n_recovery: 0`, and all eight ranks emit `rank.finalize`. The log holds 18 `rank.init` events for 8 ranks and `total_reattachments: 10`, so the table is showing a stale incarnation of a rank that detached and re-attached, not a failure. I mention it because a reader looking at the same screenshot would otherwise reasonably conclude the run was degraded, and because it is a liveness-rendering artefact of the dashboard rather than anything in the trace.